Assurance-Lattice Receipts for Fail-Closed EVM Inference Credit:
A Reproducible Development Study

Anonymous development draft
Authorship and affiliations pending accountable human approval

30 August 2026

DEVELOPMENT DRAFT — NOT SUBMISSION READY. Independent novelty, methods, provenance, external reproduction, authorship, venue, and human-approval gates remain open.

Abstract

Blockchain-mediated AI inference needs more than evidence that computation occurred: credit activation can also depend on semantic disposition, authority, provenance, freshness, evidence origin, claim scope, and exact request–response bindings. We study a typed assurance-lattice receipt (ALR) whose EVM credit gate uses a non-compensating meet rule: every required condition must pass, while failure or unknown status rejects activation with a deterministic reason code. The development prototype comprises a separately transcribed Python oracle, a Python reference kernel, and a Solidity 0.8.24 gate. We exhaustively executed all 729 states in a six-factor, three-level model and seven prespecified binding mutations. The local development run observed 0/728 false activations, 1/1 activation of the all-pass state, 729/729 base decision-and-reason matches, 7/7 mutation rejections, and 736/736 Python-generated-vector versus Solidity matches. A second same-owner local process reproduced byte-identical core result artifacts, and a later digest-pinned hermetic producer replay passed the Python and offline Foundry paths. These findings establish only development-level conformance to the frozen finite model. Novelty remains unresolved, the protocol was not independently verified before execution, the attack set is not exhaustive, no gas comparator was frozen, and no external or field validation was performed.

Keywords: blockchain assurance; EVM; verifiable AI inference; fail-closed authorization; provenance; reproducible software evaluation

1 Introduction

Optimistic and deterministic approaches already provide mechanisms for verifiable blockchain-based machine-learning inference, including fraud-proof-style verification and public re-execution [3, 4]. Signed tool receipts can distinguish direct evidence from inference [1], while an attestation study shows that omitted freshness checks can permit replay despite otherwise valid evidence [6]. Governance-first architectures further show how deterministic execution gates, evidence chains, and explicit assumptions can be evaluated as systems properties [2].

These predecessors defeat any claim that receipts, deterministic gates, optimistic inference verification, provenance, or freshness are individually new. The narrower hypothesis examined here is whether their decision-relevant dispositions can remain typed and independently failing through an EVM receipt-to-credit transition, rather than collapsing into a compensating confidence score. This remains a novelty hypothesis pending an independently owned search challenge and feature-level reconciliation with the strongest prior art.

The development contribution is a formal fail-closed rule, a minimal Python/Solidity artifact, an exhaustive finite-state evaluation, and a provenance-preserving bundle of positive, negative, and inconclusive evidence. The study does not claim a new inference-proof primitive, semantic truth, production consensus safety, field impact, or publication readiness.

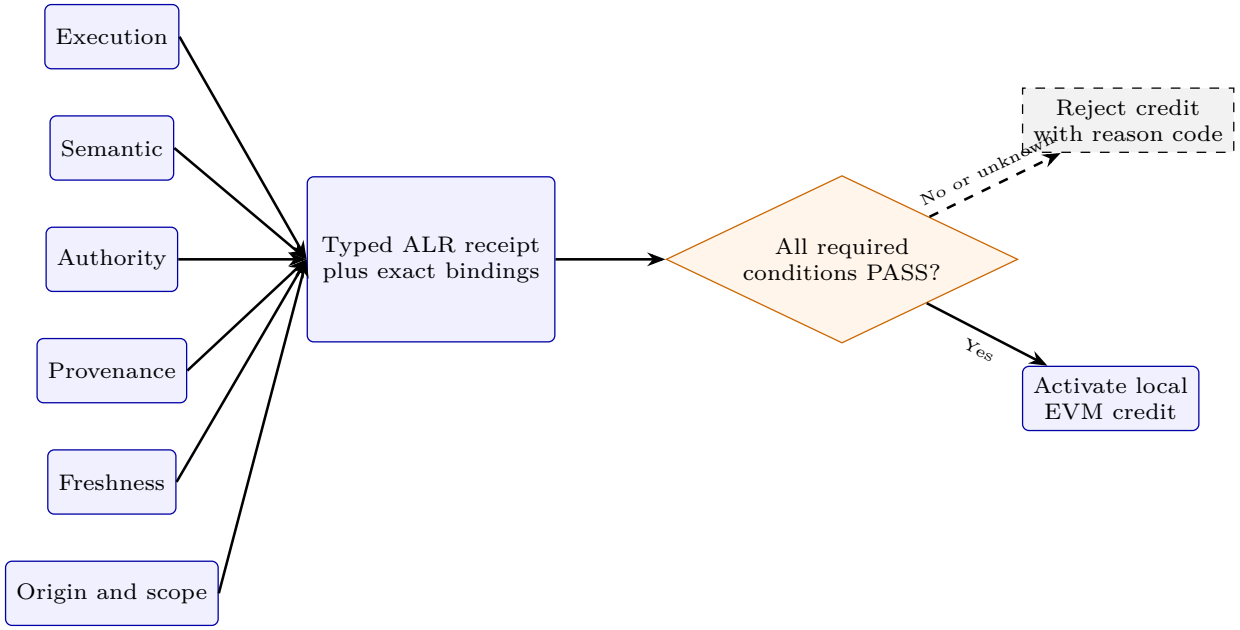

Figure 1: ALR development architecture. Six typed assurance dimensions and seven exact-binding predicates feed a non-compensating gate. Failure or unknown status rejects credit with a deterministic reason; only all-pass input activates local EVM credit. The diagram is architectural, not empirical evidence.

2 Methods

2.1 System and receipt rule

The ALR contains six assurance dimensions: execution, semantic, authority, provenance, freshness, and origin/scope. Each takes `PASS`, `FAIL`, or `UNKNOWN`. Seven exact predicates cover unused-receipt status, request and response hash bindings, authority-epoch liveness, allowed evidence origin, claim-scope agreement, and type-tag validity. Eligibility is their conjunction (Figure 1).

2.2 Deterministic gate

Algorithm 1: Fail-closed ALR eligibility.

1. For dimensions in the frozen order execution, semantic, authority, provenance, freshness, and origin/scope: return its typed `FAIL` or `UNKNOWN` reason at the first non-pass state.
2. Check, in order: receipt unused; request hash; response hash; live authority epoch; allowed evidence origin; matching claim scope; valid type tag. Return the corresponding rejection reason at the first failed predicate.
3. Return `ELIGIBLE` only if no rejection occurred.

The frozen order makes diagnostics deterministic. Consequently, reason-frequency counts reflect precedence, not empirical risk or relative dimension importance.

2.3 Design and analysis

The base design is the complete Cartesian product $3^6 = 729$, a finite population rather than a statistical sample. The all-pass tuple is the only eligible base state. Seven additional fixtures introduce one exact-binding defect at a time. An oracle transcription generates expected vectors separately from the Python reference kernel; those vectors are compiled into the Solidity test. Missing or malformed rows count as failures, and no rows are excluded or imputed. P-values and observed power are inapplicable.

Table 1: Development conditions and analysis populations.

Condition	Definition	n	Excluded
Base	Complete six-dimension, three-level Cartesian product	729	0
Mutations	One failed binding with all six dimensions passing	7	Compound faults
Replay	Core artifacts compared across two same-owner processes	3	Timing logs

Table 2: Prespecified development outcomes. Intervals show finite-population observed values, not sampling confidence intervals.

Outcome	Numerator	Denominator	Development decision
False activations among ineligible states	0	728	Pass
Activation of all-pass state	1	1	Pass
Base decision and reason agreement	729	729	Pass
Mutation rejection	7	7	Pass
Python-generated vector/Solidity agreement	736	736	Pass

2.4 Implementation and reproducibility

The run used Python 3.14.6, Foundry 1.5.1-stable, and Solidity 0.8.24 on macOS 15.1 arm64. The runner captured argv, return codes, logs, and SHA-256 values without network access. A second same-owner process replayed the workflow with a fixed protocol timestamp (Figure 2). A subsequent producer-controlled hermetic replay used a digest-pinned Linux amd64 image containing Foundry 1.5.1-v1.5.1 and exact Solidity 0.8.24. With network disabled, a read-only root and case mount, dropped capabilities, and no new privileges, it reran the Python tests, finite evaluation, generated-vector hash check, both Solidity parity tests, figures, and manuscript build. This is reproducibility evidence, not independent replication.

2.5 Ethics and evidence boundary

The evaluation used synthetic state fixtures and locally authorized existing source material. It involved no participants, personal data, live networks, or financial transactions. Institutional and venue-specific determinations remain pending accountable-human review.

3 Results

All 729 base states executed. The 728 ineligible states produced zero activations; the all-pass state activated. Expected and observed base decisions and reasons agreed for 729/729 states. All seven binding mutations rejected, and all 736 shared Python-generated vectors matched Solidity outputs (Table 2). Three Python unit tests and two Foundry tests passed.

The same-owner replay reproduced identical SHA-256 values for the base CSV, mutation CSV, and Python summary. Timing-bearing logs and run manifests differed as expected. The hermetic replay returned zero for all eight declared commands, reproduced the frozen Python hashes, passed both offline Foundry tests, and produced a venue-neutral PDF byte-identical to the earlier pinned TeX build. These checks remain producer-owned. Figure 3 reports the first-reason distribution; its 486/162/54/18/6/2/1 pattern follows the fixed precedence and is not an empirical risk ranking.

Several decisive outcomes remain negative or unresolved: independent protocol review did not precede execution; no numeric gas comparator was frozen; malformed-type parity is partial; independent reproduction and field validation were not performed; novelty is unresolved; and the prior PoI semantic result remains `NOT_SUPPORTED` with FAR 0.500 against a frozen 0.25 threshold. The prior negative is retained as context and not used as positive ALR evidence.

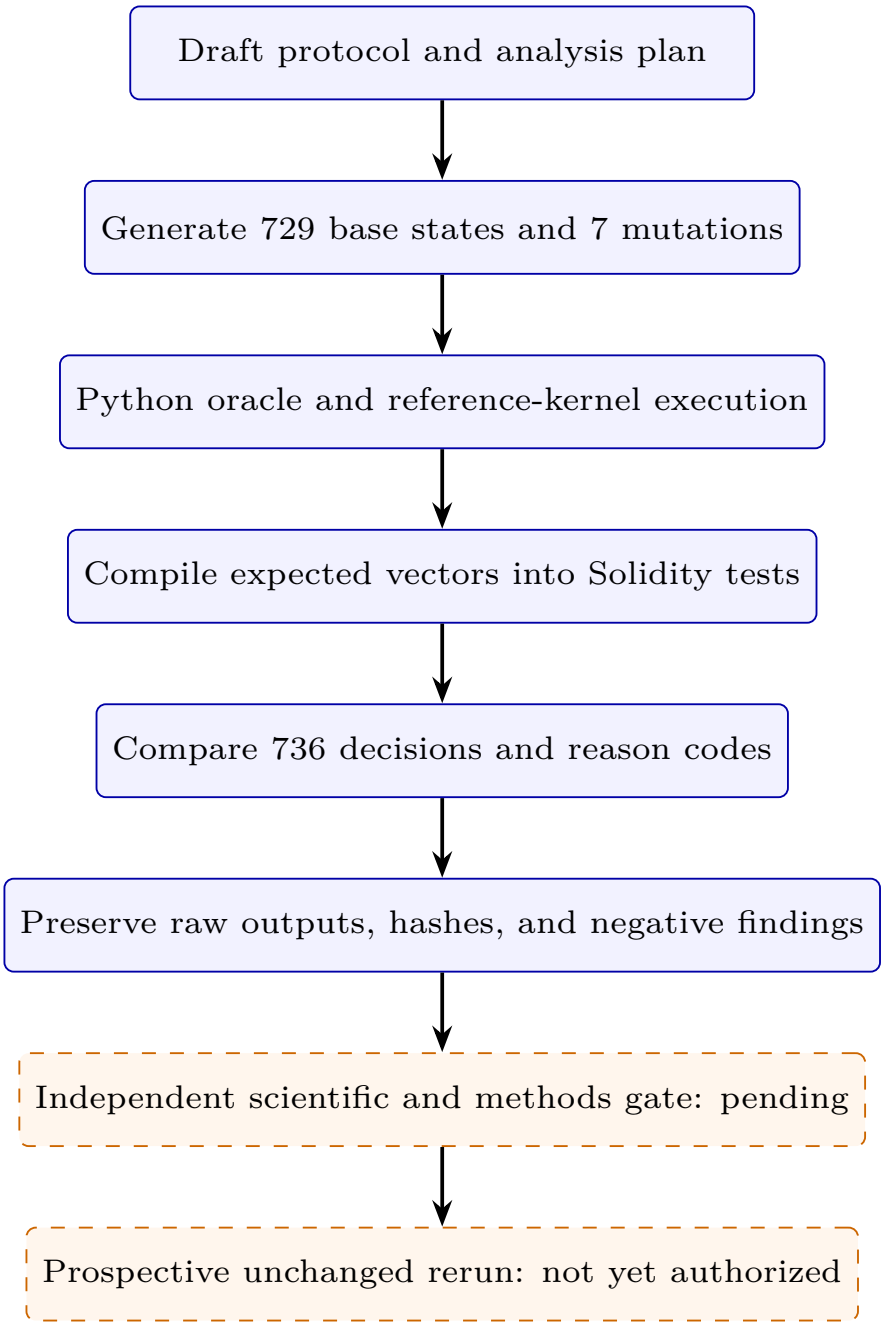

Figure 2: Development workflow and open gate. The current evidence stops before independent scientific/methods verification and a prospective unchanged rerun.

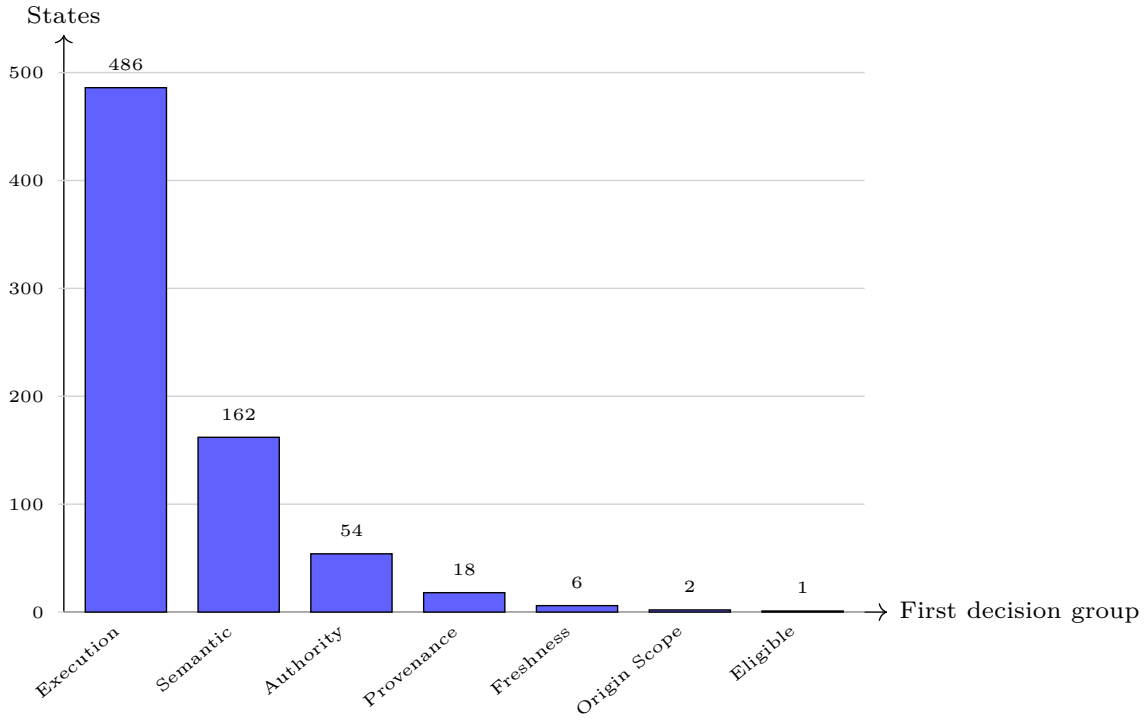

Figure 3: First-decision reason groups across the complete 729-state base matrix. Counts are deterministic products of Cartesian enumeration and the frozen precedence rule. They do not measure prevalence, severity, or assurance importance. Underlying data are in `reason-distribution-data.csv`.

4 Discussion

Within the frozen model and seven named mutations, the Python and Solidity implementations obeyed the prespecified fail-closed rule. This narrow conformance result does not establish that the model captures every assurance dimension or attack, that semantic input is correct, or that local behavior transports to production. Assurance-case research likewise cautions that successful test processes do not automatically warrant inferences from tested to fielded behavior [5].

Novelty is especially uncertain. LATTICE overlaps deterministic governance gates and auditable evidence chains; tool receipts overlap typed evidence sources; freshness work overlaps replay-resistant evidence currency; and opML/EigenAI overlap blockchain inference verification and enforcement. The remaining candidate difference is the specific typed, non-compensating EVM receipt-to-credit composition and its exhaustive development contract. Whether this is material or obvious requires independent scientific judgment.

A prospective unchanged rerun after independent protocol review is the cheapest next falsification step. A fair baseline must be frozen before any bounded-gas claim. External reproduction can then test whether the artifact and instructions suffice outside the producer's environment.

5 Conclusion

The ALR prototype provides exact development evidence for one deliberately narrow claim: within the frozen finite model and seven named binding mutations, separately implemented Python and Solidity decision paths conformed to the same fail-closed rule. That result is useful as a falsifiable software contract, but it is not evidence of semantic correctness, universal attack coverage, production safety, external generality, or material novelty. The minimum responsible publication path is therefore to preserve this claim ceiling while completing independent novelty and protocol review, a prospective unchanged rerun, and external reproduction.

6 Limitations

The protocol and analysis plan were reviewed only within the producing AI-assisted workflow. The finite model and mutation set may be incomplete. Neither the fresh-directory nor hermetic producer replay is independent replication. The derived container image is content-addressed locally but not publicly distributed. Foundry test-call gas is not transaction or production gas. The study did not evaluate semantic accuracy, decentralized adversaries, mainnet conditions, privacy, TEEs, consensus safety, user outcomes, or field impact. The bounded literature search awaits independent challenge, patent/standard refresh, and full-text predecessor reconciliation. Authorship, declarations, venue rules, and approval remain open.

7 Data and code availability

Development code, generated state tables, logs, manifests, figure data, Mermaid source, and manuscript source are preserved locally in the research-system folder. No public repository, archival DOI, license, or immutable release has been approved; public availability is therefore pending human decision, not promised.

Declarations

Authorship and CRediT: unresolved. Accountable humans must determine eligibility, order, affiliations, and roles.
Funding: unverified.
Competing interests: unverified.
Ethics: software-only development evaluation with no human participants or personal data; institutional applicability is not independently determined.
AI use: OpenAI Codex assisted with local code generation, execution, analysis, visual-source preparation, and drafting. Accountable human authors must verify all content and adapt disclosure to current venue

policy.
Submission approval: not granted.

References

[1] Abhinaba Basu. Tool receipts, not zero-knowledge proofs: Practical hallucination detection for AI agents, 2026.

[2] Elias Calboreanu. LATTICE: A governance-first architecture for authorized autonomous AI operations. *Frontiers in Artificial Intelligence*, 9:1800407, 2026.

[3] KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. opML: Optimistic machine learning on blockchain, 2024.

[4] David Ribeiro Alves, Vishnu Patankar, Matheus Pereira, Jamie Stephens, Nima Vaziri, and Sreeram Kannan. EigenAI: Deterministic inference, verifiable results, 2026.

[5] Ulysse Richard, Heather Frase, Sarah Cao, Di Cooke, Sebastian Kwon, and Adrianna Tan. Testing and evaluation of agentic AI systems in military command and control, 2026.

[6] Anton Sokolov. A challenge-nonce freshness gap in project veraison's TPM reference schemes, found by appraising application-layer action evidence end-to-end, 2026.